

JavaScript Array Method - reverse()

The `reverse()` method is one of the most commonly used array methods in JavaScript. It is used to **reverse the order of elements** in an array.

Unlike `slice()` and `concat()`, the `reverse()` method **modifies the original array**.

Table of Contents

- What is reverse()?
- Why Use reverse()?
- Syntax
- Parameters
- Return Value
- How reverse() Works
- Basic Examples
- Multiple Examples
- Real Project Examples
- Best Practices
- Common Mistakes
- Interview Questions
- Summary Table

What is reverse()?

Definition

The `reverse()` method reverses the order of the elements in an array.

It changes the **original array** and also returns the reversed array.

Why Use reverse()?

We use `reverse()` when we need to:

- Reverse a list

- Display the latest items first
- Reverse a playlist
- Reverse search history
- Process data from last to first

Syntax

```
array.reverse();
```

Parameters

The `reverse()` method does **not** take any parameters.

```
array.reverse();
```

Return Value

Returns the **same array** after reversing its elements.

The original array is modified.

How reverse() Works

Original Array

```
["Apple", "Mango", "Orange"]
```

↓

```
reverse()
```

↓

Result

```
["Orange", "Mango", "Apple"]
```

Example 1 (Basic)

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange"  
];  
  
fruits.reverse();  
  
console.log(fruits);
```

Output

```
["Orange", "Mango", "Apple"]
```

Explanation

The elements are reversed in place.

Example 2 (Numbers)

```
const numbers = [  
  10,
```

```
    20,  
    30,  
    40  
];  
  
numbers.reverse();  
  
console.log(numbers);
```

Output

```
[40,30,20,10]
```

Example 3 (Store Return Value)

```
const colors = [  
    "Red",  
    "Green",  
    "Blue"  
];  
  
const result = colors.reverse();  
  
console.log(result);  
  
console.log(colors);
```

Output

```
["Blue", "Green", "Red"]
```

```
["Blue", "Green", "Red"]
```

Both variables reference the same reversed array.

Example 4 (Single Element)

```
const items = [  
  "Laptop"  
];  
  
items.reverse();  
  
console.log(items);
```

Output

```
["Laptop"]
```

Example 5 (Empty Array)

```
const data = [];  
  
data.reverse();  
  
console.log(data);
```

Output

```
[]
```

Example 6 (Mixed Data Types)

```
const values = [  
  "JavaScript",  
  100,  
  true,  
  null  
];  
  
values.reverse();  
  
console.log(values);
```

Output

```
[null,true,100,"JavaScript"]
```

Example 7 (Objects)

```
const users = [  
  {  
    name: "John"  
  },  
  {
```

```
        name: "Sara"

    },

    {

        name: "David"

    }

];

users.reverse();

console.log(users);
```

Output

```
[
  { name: "David" },
  { name: "Sara" },
  { name: "John" }
]
```

Example 8 (Reverse Without Changing Original Array)

```
const numbers = [

    10,

    20,

    30,

    40

];
```

```
const reversed = numbers.slice().reverse();

console.log(reversed);

console.log(numbers);
```

Output

```
[40,30,20,10]

[10,20,30,40]
```

Explanation

- `slice()` creates a copy.
- `reverse()` reverses the copied array.
- The original array remains unchanged.

Real Project Example 1

Latest Notifications First

```
const notifications = [

  "Notification 1",

  "Notification 2",

  "Notification 3"

];

notifications.reverse();

console.log(notifications);
```


Output

```
[
  "Notification 3",
  "Notification 2",
  "Notification 1"
]
```

Real Project Example 2

Chat Messages

```
const messages = [
  "Hello",
  "How are you?",
  "Good Night"
];

messages.reverse();

console.log(messages);
```

Output

```
[
  "Good Night",
  "How are you?",
  "Hello"
]
```

Real Project Example 3

Playlist

```
const playlist = [  
  "Song A",  
  "Song B",  
  "Song C"  
];  
  
playlist.reverse();  
  
console.log(playlist);
```

Output

```
[  
  "Song C",  
  "Song B",  
  "Song A"  
]
```

Real Project Example 4

Student Ranking

```
const rankings = [  
  "Rahim",  
  "Karim",  
  "Rokon"  
];
```

```
rankings.reverse();  
  
console.log(rankings);
```

Output

```
[  
  "Rokon",  
  "Karim",  
  "Rahim"  
]
```

Real Project Example 5

Browser History

```
const history = [  
  "Google",  
  "YouTube",  
  "GitHub"  
];  
  
history.reverse();  
  
console.log(history);
```

Output

```
[  
  "GitHub",  
  "YouTube",  
  "Google"
```

```
"YouTube",  
"Google"  
]
```

Best Practices

Use `reverse()` when you intentionally want to modify the original array.

Create a copy if you want to preserve the original array.

```
const reversed = array.slice().reverse();
```

Use meaningful variable names.

```
reversedList  
  
latestMessages  
  
recentNotifications
```

Common Mistakes

Mistake 1

Thinking `reverse()` creates a new array.

Wrong

```
const arr = [1,2,3];  
  
arr.reverse();  
  
console.log(arr);
```

Output

```
[3,2,1]
```

The original array changes.

Mistake 2

Expecting the original array to remain unchanged.

Wrong

```
const original = [1,2,3];  
  
const reversed = original.reverse();
```

Both variables now reference the same reversed array.

Mistake 3

Not using `slice()` when copying.

Correct

```
const reversed = array.slice().reverse();
```

Mistake 4

Using `reverse()` for sorting.

```
reverse()
```

does **not** sort values.

Wrong expectation

```
[3,1,2].reverse();
```

Output

```
[2,1,3]
```

If you want ascending or descending order, use

```
sort()
```

Interview Questions

What does `reverse()` do?

It reverses the order of array elements.

Does `reverse()` modify the original array?

Yes.

What does `reverse()` return?

The reversed array.

Does `reverse()` take any parameters?

No.

How can you reverse an array without modifying the original?

```
const reversed = array.slice().reverse();
```

Which method copies an array?

```
slice()
```

Does `reverse()` sort an array?

No.

Use

```
sort()
```

for sorting.

Summary Table

Feature	Description
Method	<code>reverse()</code>
Purpose	Reverse the order of elements
Parameters	None
Returns	Reversed array
Changes Original Array	Yes
Common Uses	Notifications, Chat, Playlist, History

Quick Comparison

Method	Purpose	Returns	Changes Original Array
<code>reverse()</code>	Reverse order	Reversed Array	Yes
<code>slice()</code>	Copy array	New Array	No
<code>sort()</code>	Sort array	Sorted Array	Yes

Final Notes

The `reverse()` method is a simple but powerful array method used to display data in the opposite order. It is commonly used in:

- Chat applications
- Notification systems
- Browser history
- Music playlists
- Activity feeds
- Timeline displays

Mastering `reverse()` will help you manipulate array order efficiently and prepare you for advanced data handling in JavaScript.